

Desenvolvimento Web 2

Aula 2

Funções, Closures e Módulos

`l.bertholdo@ifsp.edu.br`

Conteúdo

- Funções
 - Funções Declaradas
 - Funções Anônimas
 - Funções Anônimas e Dinâmicas
 - Funções Autoinvocadas
 - Retorno de Objetos
- Closures
- Módulos

Funções

- Funções são blocos de código usados para definir tarefas que se repetem em diferentes lugares de um programa.
- Em vez de repetir o bloco de código da função em vários lugares, cria-se uma referência a ela, atribuindo-lhe um nome ou designando uma variável para representá-la.
- A ECMAScript 5 (ES5) prevê três sintaxes para criar funções:

Função declarada (com nome):

```
function nomeFuncao() {  
  // corpo da função  
}
```

Função anônima:

```
var varFuncao =  
function() {  
  // corpo da função  
};
```

Função anônima e dinâmica:

```
var varFuncao = new Function  
(parâmetros, corpo da função);
```

*Funções anônimas (dinâmicas ou não) também podem ser passadas como parâmetro de outra função. Nesse caso, a função é chamada de **callback** e não é preciso atribuí-la a uma variável.*

Funções Declaradas

Clique nos botões para alterar a cor do fundo da página:

Padrão

Vermelho

Azul

- Uma **função declarada** é criada definindo-se explicitamente um nome para ela. Exemplo usando a sintaxe **ES5**:

```
<body>
  <h3>Clique nos botões para alterar a cor do fundo da página:</h3>
  <button type="button" class="cor">Padrão</button>
  <button type="button" class="cor">Vermelho</button>
  <button type="button" class="cor">Azul</button>
  <script src="js/funcao-declarada.js"></script>
</body>
```

document.querySelectorAll(<seletor>):

Retorna um *array* com todos os elementos HTML do documento, conforme o seletor definido.

addEventListener(<evento>, <função>):

Adiciona um *listener* de eventos a um elemento.

Quando o evento especificado ocorre no elemento, a função é chamada.

```
var btPadrao = document.querySelectorAll(".cor")[0];
var bt1 = document.querySelectorAll(".cor")[1];
var bt2 = document.querySelectorAll(".cor")[2];
```

```
btPadrao.addEventListener("click", corPadrao); // Ao clicar em btPadrao, chama a função corPadrao().
bt1.addEventListener("click", cor1);           // Ao clicar em bt1, chama a função cor1().
bt2.addEventListener("click", cor2);           // Ao clicar em bt2, chama a função cor2().
```

```
function corPadrao() { document.body.style.backgroundColor = "white"; }
function cor1() { document.body.style.backgroundColor = "red"; }
function cor2() { document.body.style.backgroundColor = "blue"; }
```

Funções Anônimas

Clique nos botões para alterar a cor do fundo da página:

Padrão

Vermelho

Azul

- Uma **função anônima** é criada sem que se defina um nome para ela. Exemplo usando a sintaxe **ES5**:

```
<body>
  <h3>Clique nos botões para alterar a cor do fundo da página:</h3>
  <button type="button" class="cor">Padrão</button>
  <button type="button" class="cor">Vermelho</button>
  <button type="button" class="cor">Azul</button>
  <script src="js/funcao-anonima.js"></script>
</body>
```

```
var btPadrao = document.querySelectorAll(".cor")[0];
var bt1 = document.querySelectorAll(".cor")[1];
var bt2 = document.querySelectorAll(".cor")[2];

var corPadrao = function () { (document.body.style.backgroundColor = "white") };
var cor1 = function () { (document.body.style.backgroundColor = "red") };
var cor2 = function () { (document.body.style.backgroundColor = "blue") };

btPadrao.addEventListener("click", corPadrao);
bt1.addEventListener("click", cor1);
bt2.addEventListener("click", cor2);
```

Funções anônimas não são contempladas pelo mecanismo de **hoisting**, de modo que possam ser usadas antes de serem declaradas. Por este motivo, aqui as funções são definidas antes de serem chamadas pelo método **AddEventListener**.

Funções Anônimas

Clique nos botões para alterar a cor do fundo da página:

Padrão

Vermelho

Azul

- Exemplo usando a sintaxe *arrow function* da ES6:

```
<body>
  <h3>Clique nos botões para alterar a cor do fundo da página:</h3>
  <button type="button" class="cor">Padrão</button>
  <button type="button" class="cor">Vermelho</button>
  <button type="button" class="cor">Azul</button>
  <script src="js/funcao-anonima-arrow.js"></script>
</body>
```

```
let btPadrao = document.querySelectorAll(".cor")[0];
let bt1 = document.querySelectorAll(".cor")[1];
let bt2 = document.querySelectorAll(".cor")[2];
```

```
const corPadrao = () => (document.body.style.backgroundColor = "white");
const cor1 = () => (document.body.style.backgroundColor = "red");
const cor2 = () => (document.body.style.backgroundColor = "blue");
```

```
btPadrao.addEventListener("click", corPadrao);
bt1.addEventListener("click", cor1);
bt2.addEventListener("click", cor2);
```

A ES6 criou uma sintaxe simplificada para escrever funções anônimas em JavaScript:

- Foi abolida a palavra **function**.
- Antes da seta:** Parâmetros de entrada da função. Nas funções que admitem um único parâmetro, o sinal de parênteses envolvendo o parâmetro é opcional.
- Depois da seta (=>):** Script a ser processado quando a função for invocada, incluindo a saída da função e seu valor de retorno (se houver).

Funções Anônimas

- Exemplo de função anônima passada como parâmetro para outra função:

```
<body>
  <h3>Calcule o perímetro e a área de um retângulo:</h3>
  <input type="number" placeholder="Informe a base">
  <input type="number" placeholder="Informe a altura">
  <button type="button" id="calcular">Calcular</button>
  <div id="resultado"></div>
  <script src="js/funcao-anonima-callback.js"></script>
</body>
```

Calcule o perímetro e a área de um retângulo:

document.querySelector(<seletor>): Retorna o 1º elemento HTML do documento, conforme o seletor definido, no caso, os ids **calcular** e **resultado**.

Aqui, a função anônima é definida como parâmetro do método **addEventListener**. Embora proporcione um código mais conciso, esta sintaxe pode dificultar a depuração do script, devido à função não ter um nome.

```
var bt = document.querySelector("#calcular");
var rs = document.querySelector("#resultado");

bt.addEventListener("click", function () {
  var base = Number(document.querySelectorAll("input")[0].value);
  var altura = Number(document.querySelectorAll("input")[1].value);
  var perimetro = 2 * (base + altura);
  var area = base * altura;
  rs.innerHTML = `Perímetro: ${perimetro}<br />Área: ${area}`;
});
```

Em JavaScript, por padrão, campos de formulário retornam strings. Como os valores dos campos serão usados para fazer cálculos, então é preciso convertê-los para números, o que é feito por meio do método **Number** e da propriedade **value**.

A **ES6** possui um recurso chamado **Template Literals** que usa o sinal de crase (```) para concatenar strings. Essa sintaxe permite a **interpolação** de variáveis dentro de strings de forma mais limpa do que no modo tradicional que utiliza sinais de adição (+).

Funções Anônimas e Dinâmicas

- Funções declaradas com o uso do construtor **Function** são denominadas **funções anônimas e dinâmicas**. Elas permitem criar funções anônimas dinamicamente em tempo de execução a partir de strings de código.
- Essa forma de declaração é menos usada e sua sintaxe é a seguinte:

```
new Function(<"param_1", ..., "param_n">, <"corpo_funcao">);
```

```
<body>
  <h3>Construtor Function()</h3>
  <button type="button" onclick="alert('10 + 5 = ' + soma(10, 5));">
    Executar função
  </button>
  <script>
    var soma = new Function("a", "b", "return a + b");
  </script>
</body>
```

Construtor Function()

Executar função

Essa página diz

10 + 5 = 15

Funções Autoinvocadas

- Uma função autoinvocada, também chamada de IIFE (*Immediately Invoked Function Expression*), é uma função chamada automaticamente no momento em que é declarada.
- Ela cria um escopo fechado de modo que as variáveis declaradas dentro dela não ficam visíveis para o escopo global, o que torna o script mais seguro. Sua sintaxe é a seguinte:

Funções autoinvocadas são sempre envolvidas por parênteses.

```
(function(parâmetros) { //corpo da função })(valores_parâmetros); // Sintaxe ES5  
((parâmetros) => { //corpo da função })(valores_parâmetros); // Sintaxe ES6
```

```
// Exemplo com a sintaxe ES5:  
let a = 3, b = 5, resultado;  
resultado = (function (n1, n2) {  
    return (n1 + n2);  
})(a, b);
```

```
// Exemplo com a sintaxe ES6:  
let a = 5, b = 5, resultado;  
resultado = ((n1, n2) => {  
    return (n1 + n2)  
})(a, b);
```

Retorno de Objetos

- Funções podem precisar retornar mais de um valor. Nesse exemplo, a função tenta retornar dois valores. Contudo, o primeiro **return** encerra a execução da função, de modo que o segundo **return** não é executado:

```
const PI = 3.1416;  
function calculaCirculo(raio) {  
  const circunferencia = 2 * PI * raio;  
  const areaCirculo = PI * raio**2; // raio elevado a 2  
  return circunferencia; // Retorna um valor  
  return areaCirculo; // Código inalcançável ←  
}
```

- Para que a função possa retornar **mais de um valor**, pode-se definir um objeto genérico como retorno. Isso pode ser feito por meio de um **array indexado** ou de um **array associativo**.

Retorno de Objetos

- Reescrevendo o exemplo anterior para retornar os dois valores por meio de um **array indexado**:

```
const PI = 3.1416;
function calculaCirculo(raio) {
  const circunferencia = 2 * PI * raio;
  const areaCirculo = PI * raio**2; // raio elevado a 2
  → resultados = [circunferencia, areaCirculo]; // Objeto array indexado
  return resultados; // Retorna o array
}
calculaCirculo(3); // Executa a função passando raio = 3
console.log(resultados); // [18.8496, 28.2724]
console.log("Circunferência: " + resultados[0]); // 18.8496
console.log("Área do círculo: " + resultados[1]); // 28.2744
```

Retorno de Objetos

- Reescrevendo o exemplo anterior para retornar os dois valores por meio de um **array associativo**:

```
const PI = 3.1416;
function calculaCirculo(raio) {
  const circunferencia = 2 * PI * raio;
  const areaCirculo = PI * raio**2; // raio elevado a 2
  [ resultados = {
    circ: circunferencia,
    area: areaCirculo
  }; // Objeto array associativo
  return resultados; // Retorna o objeto
}
calculaCirculo(3); // Executa a função passando raio = 3
console.log(resultados); // {circunferência: 18.8496, area: 28.2724}
console.log("Circunferência: " + resultados.circ); // 18.8496
console.log("Área do círculo: " + resultados.area); // 28.2744
```

Closures

- Em JavaScript, o termo ***closure*** (clausura) está relacionado à ação de **aninhar** uma **função (interna)** dentro de outra **função (externa)**.
- Funções aninhadas rodam dentro da função na qual estão aninhadas. Assim, uma **função interna** é visível somente dentro de sua **função externa** e não será executada se for invocada fora dela. Exemplo:

```
<body>
  <h3>Closure - Exemplo 1</h3>
  <button type="button" onclick="funcaoExterna()">Função Externa</button>
  <button type="button" onclick="funcaoInterna()">Função Interna</button>
  <script>
    function funcaoExterna() {
      alert("Função Externa");
      function funcaoInterna() {
        alert("Função Interna");
      }
    }
  </script>
</body>
```

A **função externa** é executada normalmente, mas a **interna** não é executada, pois foi chamada fora da função externa.

Closure - Exemplo 1

Função Externa

Função Interna

Essa página diz

Função Externa

✖ ▶ Uncaught ReferenceError: funcaoInterna is not defined
at HTMLButtonElement.onclick (05-funcao-closure-01.html:11:53)

[05-funcao-closure-01.html:11](#)

Closures

- Neste exemplo, a **função interna** é chamada dentro da **função externa**:

```
<body>
  <h3>Closure - Exemplo 2</h3>
  <button type="button" onclick="funcaoExterna()">Função Externa</button>
  <script>
    function funcaoExterna() {
      alert("Função Externa");
      function funcaoInterna() {
        alert("Função Interna");
      }
      funcaoInterna();
    }
  </script>
</body>
```

A **função interna** é executada, pois foi chamada dentro da **função externa**.

Closure - Exemplo 2

Função Externa

Essa página diz

Função Externa

OK

Essa página diz

Função Interna

OK

Closures

- É possível escapar de uma *closure*, criando uma **referência global** para a **função interna**.

Closure - Exemplo 3

Função Externa

Escapa Closure

Essa página diz
Função Externa

Essa página diz
Função Interna

```
<body>
  <h3>Closure - Exemplo 3</h3>
  <button type="button" onclick="funcaoExterna()">Função Externa</button>
  <button type="button" onclick="escapaClosure()">Escapa Closure</button>
  <script>
    let escapaClousure; // Variável global para a função interna
    function funcaoExterna() {
      alert("Função Externa");
      function funcaoInterna() {
        alert("Função Interna");
      }
      escapaClosure = funcaoInterna; // Cria uma referência para a função interna
    }
  </script>
</body>
```

Primeiro é preciso criar a variável global **escapaClosure**. Ao chamar a função externa, esta variável recebe uma referência da função interna (**escapaClosure = funcaoInterna**), tornando-a disponível no escopo global.

Ao clicar no botão que chama a função externa (**Função Externa**) e, a seguir, no botão que chama a função interna (**Escapa Closure**), os dois alerts serão apresentados. No entanto, se o usuário clicar primeiro no botão **Escapa Closure**, seu respectivo alerta não será apresentado. Isso acontece porque a **função interna** fica disponível no escopo global somente após a execução da **função externa**.

Closures

- Um exemplo comum de *closure* ocorre em uma estrutura de repetição **for**:

```
<body>
  <h3>Closure - Exemplo 4</h3>
  <button type="button">Botao 0</button>
  <button type="button">Botao 1</button>
  <button type="button">Botao 2</button>
  <script>
    const botao = document.querySelectorAll("button");
    const botaoClicado = function () {
      alert("Você clicou no botão " + i);
    };
    for (var i = 0; i < botao.length; i++) {
      botao[i].addEventListener("click", botaoClicado);
    }
  </script>
</body>
```

Primeiro, os três botões da página são armazenados no *array* **botao**. Após definir a função, é criado um *loop* para adicionar um *listener* do evento **click** a cada botão. Com isso, ao clicar em um botão, espera-se que a função retorne o número do botão clicado.

Closure - Exemplo 4

Botao 0 Botao 1 Botao 2

Essa página diz

Você clicou no botão 3

Esse script não funciona como esperado, pois qualquer que seja o botão clicado, o alerta sempre é "Você clicou no botão 3". Isso ocorre porque os valores de *i* de cada iteração ficaram "presos" dentro da closure do bloco **for**, ou seja, apenas o último valor de *i* (3) pode ser acessado pela função **botaoClicado**.

Closures

- Para escapar da *closure* do bloco **for**, uma solução é criar um escopo próprio para a variável de incremento usando uma **função autoinvocada**:

```
<body>
  <h3>Closure - Exemplo 5</h3>
  <button type="button">Botao 0</button>
  <button type="button">Botao 1</button>
  <button type="button">Botao 2</button>
  <script>
    const botao = document.querySelectorAll("button");
    function botaoClicado(j) {
      return function () {
        alert("Você clicou no botão " + j);
      };
    }
    for (var i = 0; i < botao.length; i++) {
      botao[i].addEventListener("click", botaoClicado(i));
    }
  </script>
</body>
```

Essa função autoinvocada cria um escopo próprio para a variável **i** do bloco **for**. Com isso, cada botão passa a retornar seu respectivo valor corretamente.

Closure - Exemplo 5

Botao 0 Botao 1 Botao 2

Essa página diz

Você clicou no botão 0

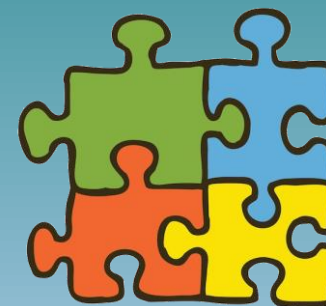
Essa página diz

Você clicou no botão 1

Essa página diz

Você clicou no botão 2

Módulos



- A **modularização do código** facilita a manutenção, a correção de *bugs*, a reutilização e a colaboração em projetos grandes e complexos.
- Em JavaScript, os arquivos de scripts que possuem **funções e variáveis exportáveis** são chamados de **módulos**.
- Criar e usar módulos possibilita organizar e estruturar o código JavaScript em arquivos separados, permitindo que diferentes partes do código possam ser **exportadas** e **importadas**.
- Nesse contexto, a **ES6** introduziu as diretivas **export**, para definir módulos, e **import**, para importá-los.

Módulos

- No exemplo a seguir, uma **página HTML** usa o script **main.js**, que por sua vez importa o script **funcoes.js**.

```
// funcoes.js
var revisar = function(revisao) { return console.log(revisao); };
var aprovar = function(aprovacao) { return console.log(aprovacao); };
export default revisar;
export {aprovar};
```

A diretiva **export** define as funções **revisar** e **aprovar** do script **funcoes.js** como **exportáveis**.

*Normalmente, declara-se a diretiva **export**, com as funções a serem exportadas, no final do arquivo.*

*No exemplo, existem duas sintaxes de exportação das funções: uma com uso da palavra **default** e outra é com uso de **chaves**.*

*A exportação com uso de **default**, em geral, é para a função principal existente no arquivo exportável. Se houver mais de uma função exportável no módulo, usa-se a sintaxe de **chaves**, na qual é possível declarar os nomes de várias funções separadas por vírgula, por exemplo: **export {aprovar, rejeitar, concluir}**.*

Módulos

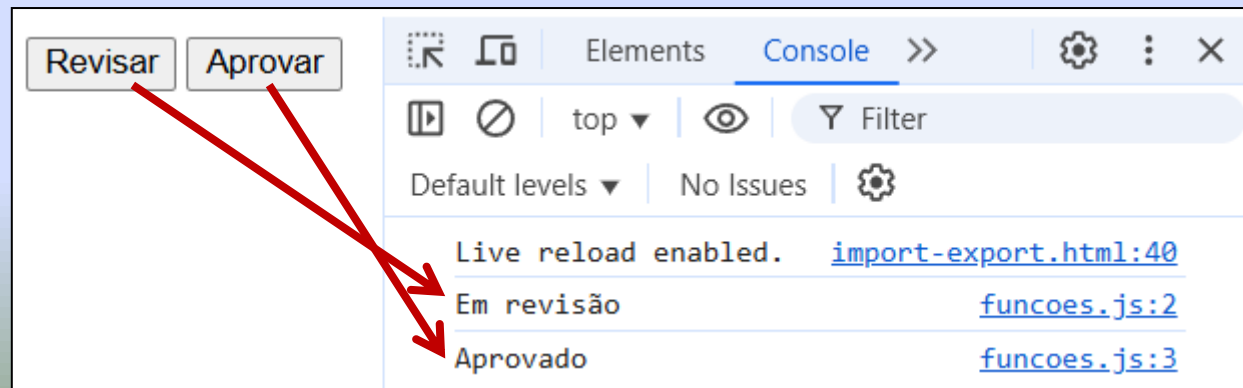
```
// main.js
import revisar from "./funcoes.js";
import aprovar from "./funcoes.js";
const bt1 = document.querySelector("#bt1");
bt1.addEventListener("click", function () {
  revisar("Em revisão");
});
const bt2 = document.querySelector("#bt2");
bt2.addEventListener("click", function () {
  aprovar("Aprovado");
});
```

```
<body>
  <button id="bt1">Revisar</button>
  <button id="bt2">Aprovar</button>
  <script type="module" src="js/main.js"></script>
</body>
```

Ao vincular ao HTML arquivos JavaScript que importam scripts, deve-se usar o atributo **type="module"** na **tag script**.

A diretiva **import** faz com que o script **funcoes.js** possa ser usado dentro do script **main.js**. O 1º import importa a função *default* (**revisar**) e o 2º importa a função não *default* (**aprovar**).

Os scripts exportados e importados precisam ser executados em um **servidor local**, ou seja, abrir o arquivo HTML diretamente no navegador não funcionará. Caso esteja usando o **Visual Studio Code**, uma sugestão é instalar a extensão **Live Server**. Depois de instalá-la, basta clicar no arquivo HTML e selecionar **Open with Live Server** para abri-lo no browser padrão configurado.



Módulos

- Além de funções, módulos podem conter **variáveis exportáveis**. No exemplo a seguir, uma **página HTML** usa o script **main.js**, que por sua vez importa o script **variaveis.js**.

```
// variaveis.js
export var idade = 28;
export const pi = 3.1416;
export const nivel = ['baixo', 'médio', 'alto'];
```

OU

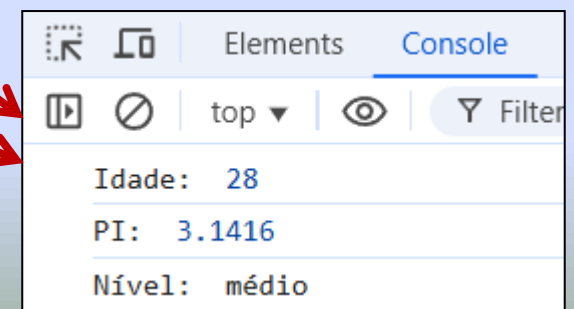
```
// variaveis.js
var idade = 28;
const pi = 3.1416;
const nivel = ['baixo', 'médio', 'alto'];
export {idade, pi, nivel};
```

```
// main.js
import {idade, pi, nivel} from "./variaveis.js";
console.log("Idade: ", idade);
console.log("PI: ", pi);
console.log("Nível: ", nivel[1]);
```

OU

```
// main.js
import * as v from "./variaveis.js";
console.log("Idade: ", v.idade);
console.log("PI: ", v.pi);
console.log("Nível: ", v.nivel[1]);
```

```
<body>
  <script type="module" src="js/main.js"></script>
</body>
```



Referências

- SILVA, Maurício S. **JavaScript Essencial**. São Paulo: Editora Novatec, 2024.